

² Le but de cette séquence est de créer des pages web dynamiques.

Nous avons pour l'instant réalisé des **pages Web statiques**, c'est-à-dire que leur contenu est figé. Ici nous allons créer des pages Web dont le contenu dépend des choix de l'utilisateur du site, on parle alors de **page dynamique**.

1. Liens utiles

- Serveur XAMPP : <https://www.apachefriends.org/>
- Toute la documentation sur PHP : <http://php.net/manual/fr>
- Des bons cours et tutoriels sur PHP : <http://php.developpez.com/cours>

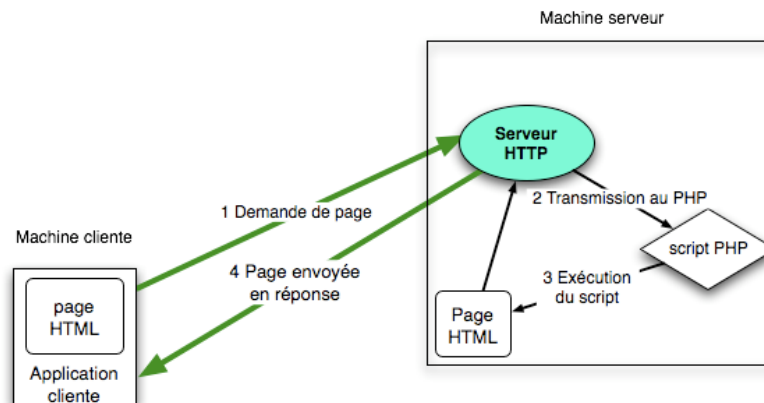
Download

Click here for other versions

XAMPP for Windows
8.2.12 (PHP 8.2.12)

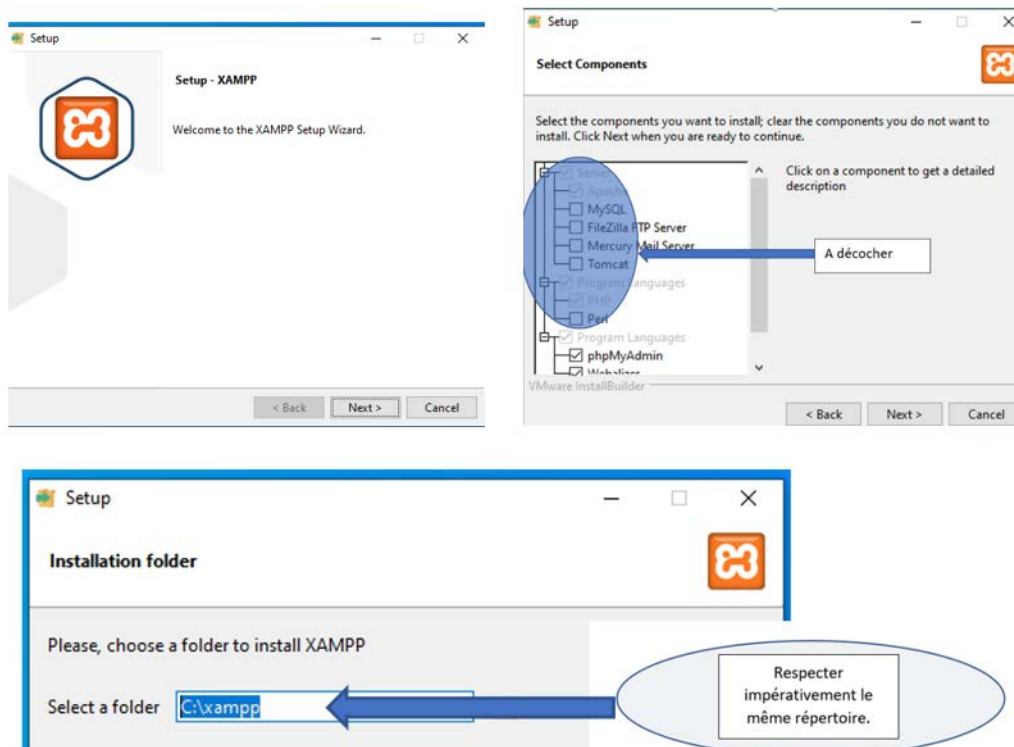
2. Installation Démarrage du serveur XAMPP

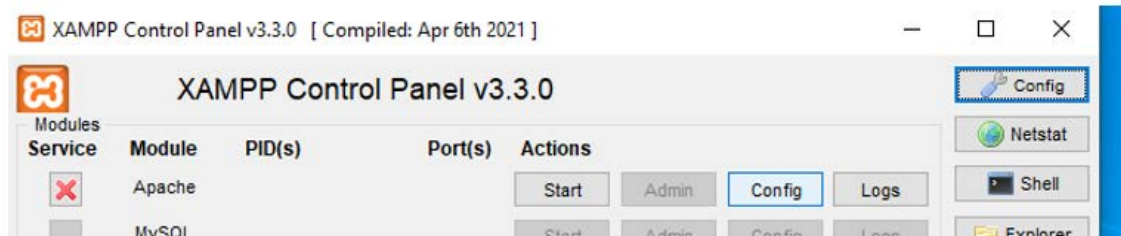
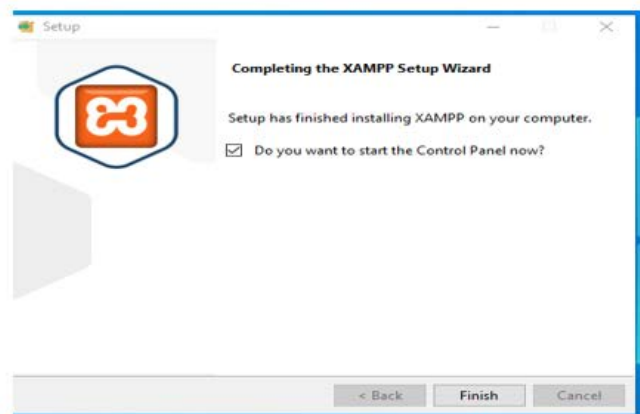
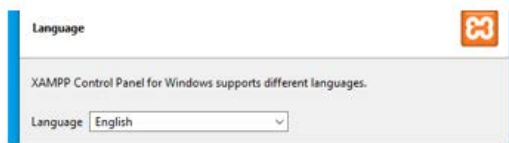
Le code PHP étant exécuté par un serveur pour générer une page HTML, un serveur Web est donc indispensable. Nous allons utiliser **XAMPP**



XAMPP est un ensemble de logiciels libres gratuits permettant de mettre en place un serveur web.

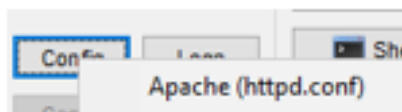
Marche à suivre pour installer XAMPP:





Si start fonctionne tant mieux sinon il faudra modifier encore certains paramètres

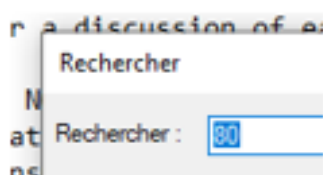
Aller dans config



Dans édition

```
Fichier  Edition  Form
#
# This is the main
# configuration d
```

Faite rechercher 80



Changer Listen 80 en 82 puis

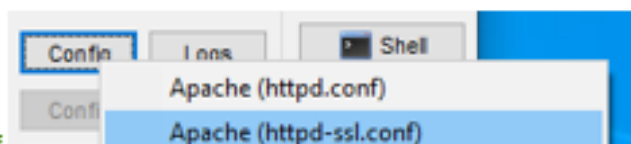
```
#Listen 12.34.56.78:80
Listen 82
```

En faisant suivant ,changer Servername localhost :80 en 82

```
# If your host doesn't ha
#
ServerName localhost:82
```

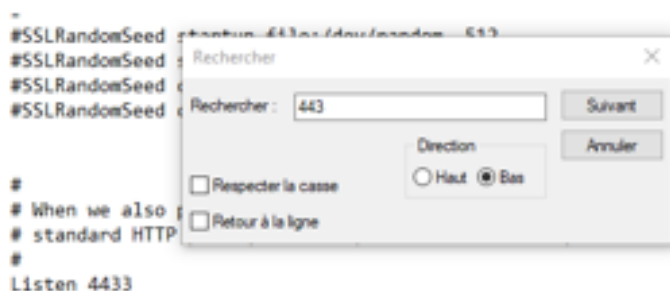
Allez un dernier effort on y arrive : sauvegarder tout cela en faisant fichier enregistrer

Faire ensuite fichier quitter



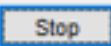
Dernière modif

Changer Listen 443 en Listen, 4433



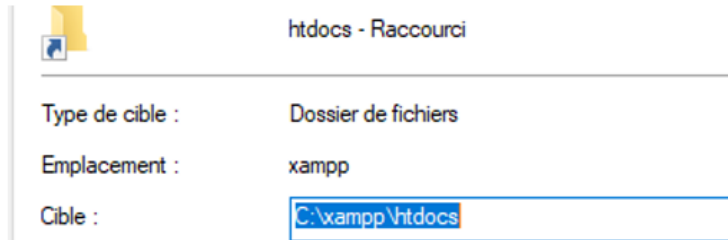
Sauvegarder tout cela en faisant fichier enregistrer

Lancer le serveur avec vos nouveau paramètres cela devrait fonctionner

Modules	Service	Module	PID(s)	Port(s)	Actions
		Apache	13544 14116	82, 4433	

On en fait quoi de ce serveur maintenant :

*Créer sur votre bureau le raccourci pour accéder plus rapidement aux fichier sauvegardé au format php dans le répertoire **XAMPP**.*



Pour toute création ou modification de fichier, utilisez Notepad++ et assurez-vous que les fichiers sont au format utf-8.

3. Premiers programmes PHP :

a) Programme BonjourToutMonde.php :

Le code PHP peut s'insérer directement dans une page Web ;

- Le code doit se trouver entre les balises **<?php** et **?>**.
- Le nom du fichier doit avoir l'extension **.php** au lieu de .html.
- Le fichier doit se trouver dans le répertoire « **C:\xampp\htdocs** ».

I) Créez un fichier nommé « **Q1.php** » et copiez dedans le contenu suivant :

```
<html>
  <head>
    <meta charset="utf-8"/>
    <title>TP PHP NSI</title>
  </head>
  <body>
    <?php
      echo("Bonjour tout le monde !");
    ?>
  </body>
</html>
```

II) Après l'avoir enregistré dans le répertoire spécifique de XAMPP.

C:\xampp\htdocs

On va maintenant ouvrir ce fichier dans **un navigateur Web** en tapant l'adresse du serveur suivi du numéro du port puis le nom du fichier (127.0.0.1 :80/Q1.php), **on peut aussi taper localhost :80/Q1.php**. Si vous avez déclaré 82 comme numéro de port , remplacer le 80 par 82

Quel est le résultat ?

Que fait la fonction echo() ?

b) Les constantes et variables :

Comme avec tous les autres langages de programmation, il est possible de créer des constantes et des variables, exemple :

```
<?php
// déclaration de constante
define('NSI', 'Numérique et Sciences Informatique');

// déclaration de variables
$nom = "Mettre votre nom ici";
$prenom = "Mettre votre prénom ici";
$age = 17; // modifiez si nécessaire
?>
<html>
<head>
    <meta charset="utf-8"/>
    <title>TP PHP NSI</title>
</head>
<body>
    <p>Je m'appelle <?php echo($prenom . " " . $nom); ?>.</p>
    <p>J'ai <?php echo($age); ?> ans.</p>
    <p>Je suis en cours de <?php echo(NSI); ?>.</p>
</body>
</html>
```

III) Copiez ce code dans un fichier nommé **Q2.php**. Placez-le au bon endroit puis ouvrez ce fichier dans un navigateur Web, quel est le résultat ?

Quel est l'opérateur de concaténation de chaîne de caractères ?

IV) Ajoutez une ligne dans **Q2.php** pour afficher une ligne supplémentaire contenant « L'année prochaine, j'aurai 18 ans. » en utilisant la variable \$age.

Pour tout savoir sur les variables : <http://php.net/manual/fr/language.variables.basics.php>

4. Le Pipotron :

Pour cette partie, télécharger les fichiers nécessaires [ici](#).

Lorsque l'on termine un rapport, un compte-rendu, un exposé,... et que l'on manque d'inspiration pour la conclusion, le Pipotron est là pour nous aider : <http://www.lepipotron.com>

Il s'agit d'un programme qui génère une phrase en assemblant 8 parties de phrase choisies par l'utilisateur ou aléatoirement. Exemple :

Eu égard à la dualité de la situation de ces derniers temps, il serait bon de s'intéresser à la totalité des voies envisageables.

Pour les 6 premières parties, il y a 10 possibilités, et 5 pour les 2 dernières parties, ce qui nous fait $10^6 \cdot 5^2$ soit 25 000 000 de conclusions possibles.

Nous allons réaliser un programme identique en PHP.

Dans un premier temps il faut entrer dans un tableau à 2 dimensions les différentes parties de la phrase. C'est ce qui est fait dans le fichier *pipo.php*.

```
<?php
//-----
    $pipo[0][0] = "Avec ";
    $pipo[0][1] = "Considérant ";
    ...
//-----
    $pipo[1][0] = "la situation ";
    $pipo[1][1] = "la conjoncture ";
```

- Le 1^{er} indice représente la partie de la phrase (de 0 à 7).
- Le 2^{ème} indice représente une des possibilités pour cette partie de la phrase (de 0 à 9).

a) Inclusion et débogage :

- La fonction `include()` permet d'inclure un autre fichier php, on s'en servira ici pour inclure *pipo.php*.
- La fonction `var_dump()` permet d'afficher les caractéristiques d'une variables (cela est bien pratique pour déboguer un programme).
- La balise html `<pre>` permet d'afficher un texte pré-formaté.

V) En utilisant ces 2 fonctions, complétez le programme **pipotronQ3.php** pour qu'il affiche la variable `$pipo`.

Tester dans le navigateur (**n'oubliez pas de placer aussi le fichier *pipo.php* dans le répertoire spécifique de XAMPP**).

b) La conclusion N° 00000000 :

On appelle la conclusion N° 00000000 celle qui utilise la 1^{ère} possibilité (indice 0) pour chaque partie de la phrase. Cette conclusion est : « Avec la situation présente, il convient d'étudier toutes les solutions envisageables. »

VI) Compléter le programme **pipotronQ4.php** pour qu'il affiche cette conclusion.

Vous noterez un petit problème de liaison (*de étudier*), nous réglerons de problème plus tard.

c) La boucle `for(;;)` :

Plutôt que de répéter 8 fois la même commande, nous allons utiliser la boucle `for(;;)` pour afficher les 8 parties de la conclusion N° 00000000.

Voir <http://php.net/manual/fr/control-structures.for.php>

VII) Compléter le programme **pipotronQ5.php** pour qu'il affiche cette conclusion N° 00000000.

Mais notre page est toujours statique ! Un petit peu d'aléatoire va rendre la page dynamique.

d) La fonction `rand()` :

Au lieu d'utiliser la valeur 0 pour le 2^{ème} indice du tableau `$pipo`, nous allons créer un nombre aléatoire entre 0 et 9.

La fonction `rand()` génère et renvoie un nombre aléatoire, voir <http://php.net/manual/fr/function.rand.php>

VIII) Compléter le programme **pipotronQ6.php** pour qu'il affiche une conclusion aléatoire.

Vérifiez que le résultat paraisse bien aléatoire.

e) Un peu d'ordre maintenant !

Pour l'instant le code PHP est incrusté un peu partout dans le code HTML. Or il est préférable de procéder d'abord aux traitements et aux calculs, et ensuite afficher le résultat. Le programme précédent va prendre la structure suivante :

```

<?php
    include("pipo.php");

    $conclusion = "";
    for ($i = ...      ) {
        $alea = ...
        $conclusion = ...
    }
?>

<html>
  <head>
    <title>Le pipotron</title>
  </head>
  <body>
    <h1>Le pipotron</h1>
    <?php echo($conclusion); ?>
  </body>
</html>

```

Partie « traitement » en PHP

```

    <?php echo($conclusion); ?>

```

Partie « affichage » en HTML.

 Dans la partie HTML, on utilisera du code PHP uniquement pour afficher des variables.

IX) Compléter le programme **pipotronQ7.php** pour qu'il affiche une conclusion aléatoire en respectant la structure ci-dessus.

f) Les fonctions

Nous avons constaté à la question VI un problème de liaison. Nous allons créer une fonction qui résout le problème.

Tout d'abord, d'où vient le problème ? Le problème survient lorsque la 4^{ème} partie de phrase se termine par « de » et que la partie suivante commence par une voyelle. Exemple :
 « Vu l'inertie actuelle, on se doit de étudier toutes les issues possibles. »

Quatre cas peuvent se produire ici :

- « de e » qu'il faudra remplacer par « d'e »
- « de é » qu'il faudra remplacer par « d'é »
- « de i » qu'il faudra remplacer par « d'i »
- « de a » qu'il faudra remplacer par « d'a »

Pour cela, nous pouvons utiliser 4 fois la fonction `str_replace()`, voir <http://php.net/manual/fr/function.str-replace.php>

Le programme aura la structure suivante :

```
<?php
    include("pipo.php");

    function adapterLiaison($str)
    {
        //
        // ici on modifie $str, à compléter
        //
        return $str;
    }

    $conclusion = "";
    for ($i ...           ) {
        $alea = ...
        $conclusion = ...
    }

    // on utilise la fonction adapterLiaison() pour supprimer
    // le problème de liaison
    $conclusion = ...
?>

<html>
    <head>
        <title>Le pipotron</title>
    </head>
    <body>
        <h1>Le pipotron</h1>
        <?php echo($conclusion); ?>
    </body>
</html>
```

X) Compléter le programme **pipotronQ8.php** pour qu'il affiche une conclusion aléatoire sans problème de liaisons.

g) Les formulaires :

Nous allons maintenant demander à l'utilisateur le N° de la conclusion qu'il souhaite obtenir, ce numéro est un nombre de 8 chiffres, chaque chiffre servira d'indice pour choisir une variante d'une partie de la phrase, exemple :

N° de conclusion	Conclusion
00000000	<i>Avec la situation présente, il convient d'étudier toutes les solutions envisageables.</i>
12345678	<i>Considérant la crise qui est la nôtre, il serait intéressant d'imaginer la totalité des problématiques s'offrant à nous.</i>
99999999	<i>Tant que durera la baisse de confiance de ces derniers temps, il faut de toute urgence se remémorer certaines alternatives de bon sens.</i>

Pour cela il nous faut insérer dans la page un formulaire. Voir <http://php.net/manual/fr/tutorial.forms.php>.

Voici le code HTML du formulaire que nous allons utiliser :

```
<form method="post" action="pipotronQ11.php">
  <label>Entrez un nombre de 8 chiffres</label>
  <input name="conclusion_no" type="number" min="0" max="99999999"/>
  <input type="submit" value="Générer" />
</form>
```

Explications :

- La 1^{ère} ligne contient la balise <form> avec 2 attributs :
 - o method : qui indique la méthode utilisée pour envoyer les données du formulaire au serveur. Il y en a 2 : GET et POST.
 - o action : indique quel fichier doit être appelé quand on cliquera sur le bouton « Générer ».
- La 2^{ème} ligne contient un <label>, c'est juste du texte à afficher.
- La 3^{ème} ligne contient une entrée du formulaire, on indique son **nom** et son **type**, en option on peut indiquer les limites pour le type « number ».
- La 4^{ème} ligne contient le bouton qui va provoquer l'envoi des données du formulaire au fichier spécifié plus haut (action="pipotronQ11.php").

XI) Insérez ce code dans le fichier précédent juste après le titre <h1>...</h1>et enregistrez sous le nom **pipotronQ9.php**
Quel est le résultat ?

Ensuite il faut récupérer les valeurs du formulaire, elles se trouvent dans la variables \$_POST.

XII) Avec la fonction var_dump(), ajouter une ligne en début de programme pour afficher cette variables.
Quel est le résultat ?

Voici ce que nous devons faire maintenant :

- Récupérer la valeur transmise par le formulaire où créer une valeur si le formulaire est vide. On utilisera une variable \$str_no.
- Ajouter des zéro à droite pour que cette \$str_no contienne 8 caractères.
- Initialiser la variable \$conclusion
- Pour chaque partie de la onclusion (il y en a 8) :
 - o Utiliser le 1^{er} caractère de \$str_no pour choisir le texte de cette partie
 - o Supprimer ce 1^{er} caractère de \$str_no
 - o Ajouter à \$conclusion la partie de conclusion sélectionnée.
- Adapter les liaisons dans \$conclusion.

Voici le pseudo-code correspondant :

```
DEBUT
  SI $_POST['conclusion_no'] est définie
  ALORS
    $str_no ← $_POST['conclusion_no']
  SINON
    $str_no ← "00000000"
  FIN SI

  TANT QUE longueur de $str_no < 8
    Ajouter le caractère '0' au début $str_no
  FIN TANT QUE

  $conclusion ← ""
  POUR $i = 0 à 7
    $j ← premier caractère de $str_no converti en nombre (de 0 à 9)
    Supprimer le 1er caractère de $str_no
    $conclusion ← $conclusion + $pipo[$i][$j]
  FIN POUR

  $conclusion ← adapterLiaison($conclusion)
FIN
```

Pour réaliser ce programme on pourra utiliser les fonctions suivantes :

- `isset()` : <http://php.net/manual/fr/function.isset.php>
- `strlen()` : <http://php.net/manual/fr/function.strlen.php>
- `intval()` : <http://php.net/manual/fr/function.intval.php>
- `substr()` : <http://php.net/manual/fr/function.substr.php>

XIII) Implémentez ce pseudo-code dans le fichier `pipotronQ9.php` et enregistrez le sous le nom `pipotronQ11.php` (pensez à modifier l'attribut « action » de la balise « form »). Testez.

5. Le poème de Raymond QUENEAU :

Dans le même genre d'idée, on peut réaliser une page Web qui affiche un des 100 000 000 000 000 de poèmes de Raymond QUENEAU.

https://fr.wikipedia.org/wiki/Cent_mille_milliards_de_po%C3%A8mes

http://emusicale.free.fr/HISTOIRE_DES_ARTS/hda-litterature/QUENEAU-cent_mille_milliards_de_poemes/cent_mille_milliards.php

XIV) Ecrire le code qui affiche le tableau 2 dimensions `poeme.php` qui est dans stocké dans une variable `$poeme`. Sauvegarder sous `poemeQ1.php` puis tester.

XV) Ecrire le code qui affiche le poème qui prendra le premier élément de chaque ligne du poème. En tout, il y a 14 lignes et on sautera une ligne supplémentaire à chaque fin de strophe (si `i=3` ou `7` ou `10` si `i` est le n° de ligne). Sauvegarder sous `poemeQ2.php` puis tester.

100 000 000 000 000 poèmes de Raymond Queneau

Le roi de la pampa retourne sa chemise
pour la mettre à sécher aux cornes des taureaux
le cornédbif en boîte empeste la remise
et fermentent de même et les cuirs et les peaux

Je me souviens encor de cette heure exeuquise
les gauchos dans la plaine agitaient leurs drapeaux
nous avions aussi froids que nus sur la banquise
lorsque pour nous distraire y plantions nos tréteaux

Du pôle à Rosario fait une belle trotte
aventures on eut qui s'y pique s'y frotte
lorsqu'on boit du maté l'on devient argentin

L'Amérique du Sud séduit les équivoques
exaltent l'espagnol les oreilles baroques
si la cloche se tait et son terlintintin

XVI) Ecrire le code qui affiche le poème qui prendra une valeur aléatoire entre (0 et 9) de chaque ligne du poème. En tout, il y a 14 lignes et on sautera une ligne supplémentaire à chaque fin de strophe (si `i=3` ou `7` ou `10` si `i` est le n° de ligne). Sauvegarder sous `poemeQ3.php` puis tester.

XVII) Ecrire le code qui affiche le poème qui prendra la valeur donnée par l'utilisateur (sous forme de nombre à 14 chiffres) de chaque ligne du poème. En tout, il y a 14 lignes et on sautera une ligne supplémentaire à chaque fin de strophe (si `i=3` ou `7` ou `10` si `i` est le n° de ligne). Sauvegarder sous `poemeQ4.php` puis tester.

100 000 000 000 000 poèmes de Raymond Queneau

Entrez un nombre de 14 chiffres

Poème généré avec le nombre 0000000000000000

Le roi de la pampa retourne sa chemise
pour la mettre à sécher aux cornes des taureaux
le cornédéf en boîte empest la remise
et fermentent de même et les cuirs et les peaux

Je me souviens encor de cette heure exequise
les gauchos dans la plaine agitaient leurs drapeaux
nous avions aussi froids que nus sur la banquise
lorsque pour nous distraire y plantions nos tréteaux

Du pôle à Rosario fait une belle trotte
aventures on eut qui s'y pique s'y frotte
lorsqu'on boit du maté l'on devient argentin

L'Amérique du Sud séduit les équivoques
exaltent l'espagnol les oreilles baroques
si la cloche se tait et son terlintintin

100 000 000 000 000 poèmes de Raymond Queneau

Entrez un nombre de 14 chiffres

Poème généré avec le nombre 12345678912345

Le cheval Parthénon s'énervé sur sa frise
pour du fin fond du nez exciter les arceaux
le chauffeur indigène attendait dans la brise
des narcisses on cueille ou bien on est des veaux

Il déplore il déplore une telle mainmise
qui clochard devenant jetait ses oripeaux
l'un et l'autre ont raison non la foule imprécise
l'enfant pur aux yeux bleus aime les berlingots

Le brave a beau crier ah cré nom saperlotte
on comptait les esprits acérés à la hotte
lorsqu'on revient au port en essayant un grain

Ne fallait pas si loin agiter ses breloques
les banquiers d'Avignon chantent-ils les baloques ?
mais on n'aurait pas vu le métropolitain

6. Javascript :

Comme nous l'avons vu, le php s'exécute du côté serveur (server-side) alors que le javascript qui a été créé en 1995 lui s'exécute du côté client (client-side).

C'est un langage orienté objet à prototype. On utilise le javascript pour modifier le html et le css pour rendre la page dynamique. Nous allons utiliser notepad++ pour écrire le code javascript mais sachez que vous pouvez tester des parties de code sur le site [jsbin](http://jsbin.com) ou [jsfiddle](http://jsfiddle.com).

a) Emplacement du code javascript

On peut écrire le code soit :

- Dans l'en-tête (head) de la page html
- Dans le corps (body) de la page html (il vaut mieux le faire à la fin du body)
- Dans un fichier à part ??? .js mais il faut indiquer dans l'en-tête le nom du fichier que l'on va utiliser

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8"/>
    <title>TP JAVA NSI</title>
  </head>
  <body>
    <h1>Cette page contient du javascript    </h1>
    <p> On peut écrire le script java à différents endroits... </p>
    <ol>
      <li>Dans l'élément head d'une page html</li>
      <li>Dans l'élément body d'une page html</li>
      <li>Dans un fichier .js séparé </li>
    </ol>

    <script>
      alert("Ceci est un code javascript dans le body") ;
    </script>
  </body>
</html>
```

XVIII) Créez un fichier nommé « JavaQ1.html » et copiez dedans le contenu ci-dessus. Que fait la commande alert ?

XIX) Déplacer le script dans l'en-tête, sauvegarder sous « JavaQ2.html » puis tester.

Il est recommandé de mettre le script dans un fichier séparé .js pour différentes raisons :

- 🔧 Meilleure performance car le fichier externe sera mis en cache et ne consommera plus de ressources
- 🔧 Plus de clarté
- 🔧 Plus de facilité de mise à jour et maintien du script pour différents projets.

XX) Créer un fichier scriptJava.js et ajouter la ligne de code :

```
alert("Ceci est un code javascript dans un fichier séparé");
```

Reste à lier le fichier html et .js, pour cela dans l'en-tête rajouter :

```
<script src="scriptJava.js">
```

```
</script>
```

Enregistrer sous JavaQ3.html et tester.

En javascript, il faut finir chaque instruction par un ;
Dès qu'une instruction appartient à une autre instruction, il faut indenter (décaler d'une tabulation l'instruction).

Un commentaire sur une ligne s'écrit avec //.

Un commentaire multilignes s'écrit /*.....

```
* .....
* .....
* .....
* .....
* .....
* .....
* .....
*/
```

b) Les variables :

C'est un conteneur qui va stocker une information. Pour créer une variable, on utilise l'instruction let suivi du nom de la variable qui suit les règles suivantes :

- 🚩 Chaque variable doit avoir un nom unique=identifieur.
- 🚩 Le nom de la variable ne peut contenir que des lettres non accentuées, des chiffres et _ ou \$.
- 🚩 Le nom des variables doit commencer par une lettre
- 🚩 Le nom est sensible au majuscules et minuscules toto≠Toto.

XX)Créer un fichier variablesJava.js et ajouter le code ci-dessous ainsi que le fichier JavaQ4.html et tester.

```
let prenom="Mettez votre prénom",nom="Mettez votre nom";//on déclare deux
variables chaine de caractères sur la même ligne en séparant par une virgule
let age=17; //C'est un nombre
let homme=true;//C'est un booléen, mettre true si vous êtes un homme ou false
sinon.
let n=null; //on ne connaît pas la valeur de la variables
let u=undefined;//la variable n'a pas été défini mais on le sait.
let na=NaN;//Not a number
let v=''; //la variable est vide.

alert("prenom : "+typeof(prenom));
alert("age : "+typeof(age));
alert("homme : "+typeof(homme));
alert("n : "+typeof(n));
alert("u : " +typeof(u));
alert("na :"+typeof(na));
alert("v :"+ typeof(v));
```

JavaQ4.html :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8"/>
    <title>TP JAVA NSI</title>
    <script src="variablesJava.js">
    </script>
  </head>
  <body>
    <h1>Cette page contient du javascript    </h1>
  </body>
</html>
```

Est-ce que tous les types correspondent à ce que vous attendiez ?

c) Opérations sur les variables :

XXI)Créer un fichier operationJava.js et ajouter le code ci-dessous et modifier JavaQ4.html afin qu'il soit lié à ce nouveau script, sauvegarder sous JavaQ5.html et tester. Que fait le signe + ?

```
let prenom="Mettez votre prénom",nom="Mettez votre nom" ;
let age=17;
let homme=true;//mettre true si vous êtes un homme ou false si une femme.
```

```
alert("Je m'appelle " + prenom + " " + nom);
```

XXII) Modifier le script operationJava.js afin qu'il affiche dans une nouvelle boîte de dialogue, « J'ai 17 ans et l'année prochaine j'aurai 18 ans ».

On peut tester des variables avec :

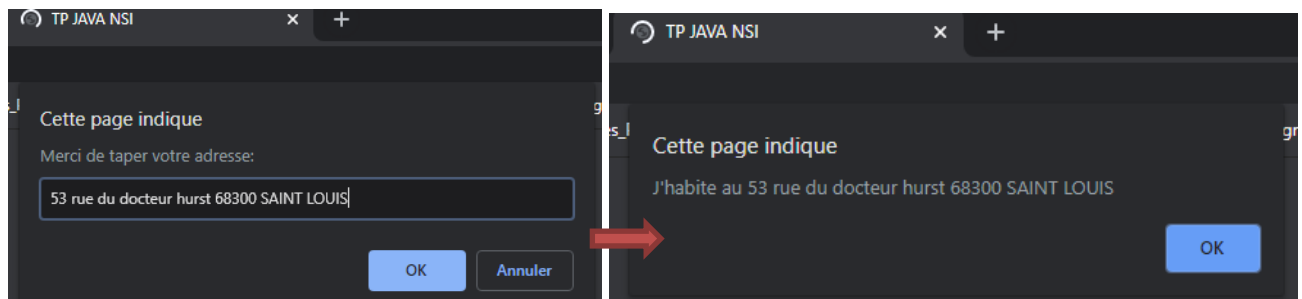
```
if (« condition »)
{
instruction1 ;
instruction 2 ;....}
else
{
instruction1 ;
instruction 2 ;....}
```

SYMBOLE	SIGNIFICATION
==	est égal à (valeur)
!=	est différent de (valeur)
===	est égal à (valeur ET type)
!==	est différent de (valeur OU type)
>	est strictement supérieur à
<	est strictement inférieur à
>=	est supérieur ou égal à
<=	est inférieur ou égal à

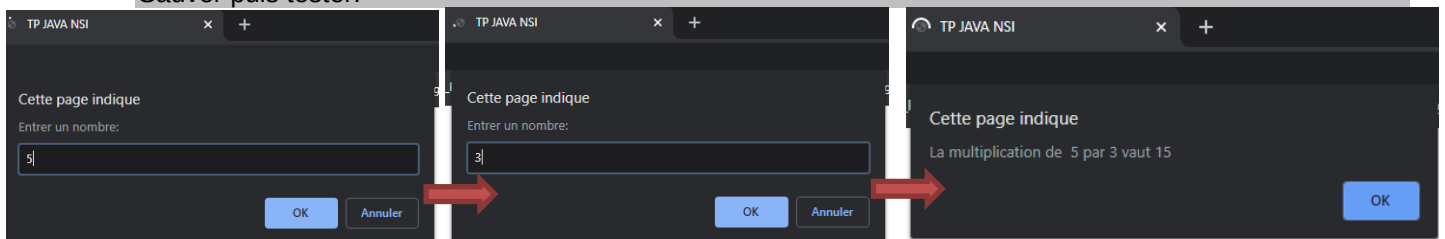
XXIII) Modifier le script operationJava.js en créant une variable sexe="un homme";
Ajouter la condition si homme == true, alors sexe = "un homme" sinon sexe "une femme".
Afficher Je suis un homme (ou Je suis une femme suivant la valeur du booléen homme).
Sauver puis tester.

Pour interagir avec l'utilisateur, on utilise la fonction prompt ("Texte à afficher :").

XXIV) Modifier le script operationJava.js en créant une variable adresse=la valeur saisie par l'utilisateur grâce à la fonction prompt("Merci de taper votre adresse:").
Afficher J'habite au « adresse ».
Sauver puis tester.



XXV) Modifier le script operationJava.js en créant deux variables x et y =la valeur saisie par l'utilisateur grâce à la fonction prompt("Entrer un nombre:").
Afficher la multiplication de « x » par « y » vaut « x*y ».
Sauver puis tester.



d) Les opérateurs logiques :

OPERATEUR LOGIQUE	SYMBOLE
ET	&&
OU	
NON	!

XXVI) Modifier le script operationJava.js en créant une variable heure =l'heure saisie par l'utilisateur grâce à la fonction prompt("Donner l'heure sous forme d'entier:").

Ajouter les instructions qui affiche :

- « cette heure n'est pas valide » si heure <0 ou >24
- « c'est la matin » si heure >0 et <12
- « c'est l'après-midi » si heure >12 et <18
- « c'est le soir » si heure >18.

On pourra utiliser else if {...}

Sauver puis tester.

On peut écrire une condition en structure ternaire.

Exemple :

```
let bon=(heure<17) ? "Bonjour" : "Bonsoir";
alert(bon)
```

est équivalent à

```
if (heure<17)
{
    alert("Bonjour") ;
}
else
{
    alert("Bonsoir") ;
}
```

Le switch est une méthode alternative qui suivant plusieurs valeurs, gère les cas (il ne gère que les égalités et non pas les inégalités).

Il faut un : à la fin de chaque cas et un break pour sortir du switch si le cas est vrai sinon il testera tous les cas.

Il faut laisser le cas default qui est celui que l'on fait si aucun cas n'est valide.

XXVII) Modifier le script operationJava.js en ajoutant et complétant le code ci-dessous.
Sauver puis tester.

```
switch(heure)
{
    case "8" :
        alert("Il est 8h");
        break;
    case "10" :
        alert("Il est 10h");
        break;
    case « A compléter » :
        alert("Il est 12h");
        break;
    case « A compléter » :
        alert("Il est 16h");
        break;
    default :
        alert("Rien à afficher pour cette valeur");
}
```

Jusqu'à présent, nous avons toujours affiché les éléments dans une boîte de dialogue. Il est possible dans l'afficher dans la page à chaque exécution.

Pour cela, il suffit de créer dans la page html un identifiant id dont on va modifier le contenu grâce à **document.getElementById(« nom de l'id »).innerText= « la nouvelle valeur à afficher »**

e) Les boucles : While, do While et for

- While(« condition »){..} : la boucle est exécutée si la condition est vraie.
- do {..} while(« condition ») : la boucle est toujours exécutée au moins une fois.
- for (let i=0 ;i< « condition » ;i++){...} : l'initialisation est exécutée avant tout passage dans la boucle et l'incréméntation et la condition seront testées après l'exécution de la boucle for.

XXVIII)Créer un fichier bouclesJava.js et ajouter le code ci-dessous ainsi que le fichier JavaQ6.html et tester.

```
let nb=prompt("Entrer la numero de la table que vous souhaitez afficher:");
document.getElementById("tab").innerText = "La table de multiplication de "
+ nb + " est : ";
let resultat_a_afficher="";
for (let i=1;i<=10;i++){
    resultat_a_afficher=resultat_a_afficher+i+" * "+nb+" = "+i*nb+"\n";
}
document.getElementById("resultat").innerText = resultat_a_afficher;
```

JavaQ6.html :

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8"/>
  <title>TP JAVA NSI</title>
  <script src="bouclesJava.js">
  </script>
</head>
<body>
  <h1>Cette page contient du javascript</h1>
  <h2 id="tab">La table de multiplication de 0 est :<br/></h2>
  <h2 id="resultat">La table va s'afficher</h2><br/>
</body>
</html>
```

Est-ce le résultat que vous attendiez ?

Le problème vient du fait que le script Java est exécuté au début puis le body va écraser ce que le script vient de faire. Il faut que le script java soit lu de façon asynchrone pour cela, rajouter async après le nom du fichier

```
<script src="bouclesJava.js" async>
```

puis sauver puis retester.

f) Les objets :

Un objet est composé de propriétés et méthodes.

Exemple : Pour créer un objet moi :

```
let moi={
  prenom="Votre prénom",
  nom="Votre nom",
  age=Votre age,
}
```

Pour accéder au prénom, on accèdera à la méthode prenom grâce à la ligne de code moi.prenom()

Nous allons créer un objet personne

```
function Personne(prenom, nom,age)
{
  this.prenom=prenom;
  this.nom=nom;
  this.age=age;
}
```

XXIX) Créer un fichier ObjetsJava.js et ajouter le code ci-dessous ainsi que le fichier JavaQ7.html et tester.

```
function Personne(prenom, nom,age)
{
  this.prenom=prenom;
  this.nom=nom;
  this.age=age;
}
let VOTRE PRENOM =new Personne("VOTRE PRENOM", "VOTRE NOM", VOTRE AGE);
let paul=VOTRE PRENOM;
paul.prenom="paul";
alert(VOTRE PRENOM.prenom);
```

Java7.html

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8"/>
  <title>TP JAVA NSI</title>
  <script src="ObjetsJava.js" async>
  </script>
</head>
<body>
  <h1>Cette page traite les objets javascript</h1>
</body>
</html>
```

Est-ce le résultat que vous attendiez ?

Attention on accède aux objets par référence. Donc, dès que l'on assigne un objet à un autre, on pointe sur le même élément donc lorsqu'on en modifie un, on modifie l'autre.

Les méthodes des objets racines :

Objet	Méthode	But
String	toLowerCase()	Change les caractères en minuscules
	toUpperCase()	Change les caractères en majuscules
	charAt(p)	Renvoie le caractère à la position p (on commence à 0)
	indexOf("c")	Renvoie la première position du caractère c que l'on recherche (renvoie -1 si non trouvé et on peut lui dire de chercher à partir d'une certaine position p en le rajoutant en paramètre indexOf("c",p)
	lastIndexOf("c")	Renvoie la dernière position du caractère c que l'on recherche
	replace("cavant","caprès")	On remplace une chaine avant par après.
	slice(p1,p2)	On récupère une partie du texte de l'indice p1 à p2. Le caractère à la position p2 est exclu.
	trim()	Supprime les espaces en début et fin de chaine de caractères
Number	isInteger(nb1)	Renvoie vrai si nb1 est un entier
	toFixed(nb)	Formate un nombre en indiquant le nombre de décimales nb
	toPrecision(nb)	Formate un nombre en indiquant le nombre de chiffres significatifs nb
Math	ceil(nb1)	Arrondit nb1 à l'entier immédiatement supérieur (ou égal).
	trunc(nb1)	Ignore la partie décimale de nb1 et retourne sa partie entière.
	floor(nb1)	Arrondit nb1 à l'entier immédiatement inférieur (ou égal).
	round(nb1)	Arrondit nb1 à l'entier le plus proche
	min(nb1,nb2,...)	Renvoie le plus petit nombre d'une série de nombres
	max(nb1,nb2,...)	Renvoie le plus grand nombre d'une série de nombres
	abs(nb1)	Renvoie la valeur absolue de nb1
	cos(nb), sin(nb), tan(nb), acos(nb), asin(nb),atan(nb)	Renvoient respectivement le cosinus, le sinus, la tangente, l'arc cosinus, l'arc sinus et l'arc tangente de nb en radians.
	exp(nb)	Renvoie l'exponentielle de nb
	log(nb)	Renvoie le logarithme népérien

g) Les tableaux: Array

Les tableaux sont avant tout des objets qui dépendent de l'objet global Array, se sont des éléments qui vont pouvoir contenir plusieurs valeurs. La première valeur du tableau a comme indice 0. Pour créer un tableau, il suffit de mettre les valeurs de la variable entre [...] séparées d'une « , ».

XXX)Créer un fichier tableauxJava.js et ajouter le code ci-dessous ainsi que le fichier JavaQ8.html et tester.

```
let prenom = ['Pierre', 'Paul', 'Jacques', 'Camille'];
let nombre = [18,16,17,20];
let produits = ['livre',20,'ordinateur',5,['cartes',52]];
document.getElementById("a").innerHTML=prenom[0]+' possède un '+produits[2];
document.getElementById("b").innerHTML=prenom[1]+' possède '+nombre[0]+ ' '
+produits[0]+'s';
document.getElementById("c").innerHTML=prenom[2]+' possède '+produits[4][1]
+' '+produits[4][0];
```

Java8.html

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8"/>
    <title>TP JAVA NSI</title>
    <script src="tableauxJava.js" async ></script>
  </head>
  <body>
    <h1>Cette page traite des tableaux javascript</h1>
    <h2 id="a"> </h2>
    <h2 id="b"> </h2>
    <h2 id="c"> </h2>
  </body>
</html>
```

A quoi correspond :

- produits[4][1]
- prenom[1]
- nombre[0].

Pour parcourir un tableau élément par élément, on va pouvoir utiliser une boucle spécialement créée dans ce but qui est la boucle for...of.

Rajouter un id = "d" sur le fichier html.

Ajouter sur le fichier javascript :

```
for (let valeur of prenom){
    document.getElementById("d").innerHTML += valeur + '<br>';
}
```

Sauver et tester. Expliquer ce que fait le code ci-dessus.

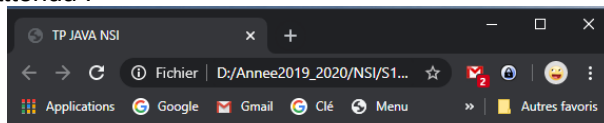
Les méthodes des tableaux :

Objet	Méthode	But
Array	push(p1,p2,...)	Ajoute les éléments p1, p2... en fin de tableau et retourne la nouvelle taille du tableau.
	pop()	Supprime le dernier élément d'un tableau et retourne l'élément supprimé
	unshift (p1,p2,...)	Ajoute les éléments p1, p2... en début de tableau et retourne la nouvelle taille du tableau.
	shift()	Supprime le premier élément d'un tableau et retourne l'élément supprimé
	splice(ideb,nb,p1,...)	Remplace à partir de l'indice ideb les nb éléments par p1..... Si p1... n'est pas donné alors on supprime simplement les nb éléments à partir de ideb
	join('sep')	Retourne une chaîne de caractères créée en concaténant les différentes valeurs d'un tableau avec comme séparateur sep. Si sep n'est pas donné alors il prendra la virgule par défaut.

XXXI)Créer un fichier exoJava.js et un fichier JavaQ9.html qui répond au cahier des charges suivant :

- une boîte de dialogue s'affiche et demande le nom de l'élève.
- une deuxième boîte s'ouvre et demande le prénom.
- une troisième boîte s'ouvre et demande la note obtenue.
- ces trois boîtes s'ouvrent tant que l'utilisateur n'a pas donné comme nom le mot « fin ».
- une fois ces valeurs rentrées pour chaque élève, ces éléments sont affichés sur la page et donne la moyenne de la classe (avec deux chiffres après la virgule).

Résultat attendu :



Cette page traite les notes d'une classe

Bareux Gisèle 15 /20
Dupont Pierre 18 /20
Titi Toto 14 /20

Moyenne de la classe = 15.67 /20

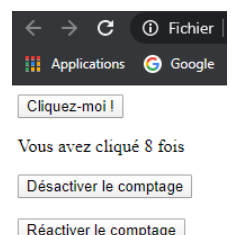
h) Les boutons:

Pour créer un bouton, il suffit d'utiliser les balises <button id= ".....">....</button>

Nous allons créer la page ci-contre :

La méthode addEventListener() d'EventTarget met en place une fonction à appeler chaque fois que l'événement spécifié est remis à la cible.

La méthode removeEventListener() supprime d'une EventTarget un écouteur d'événements précédemment enregistré avec EventTarget.addEventListener().



XXXII) Créer un fichier `compteurClics.js` et un fichier `JavaQ10.html` avec les contenus ci-dessous

```
function clic() {
    compteurClics++;
    document.getElementById("compteurClics").textContent = compteurClics;
}
var compteurClics = 0;
document.getElementById("clic").addEventListener("click", clic);
document.getElementById("desactiver").addEventListener("click", function () {
    document.getElementById("clic").removeEventListener("click", clic);
});
```

Java10.html

```
<!doctype html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Compteur de clics</title>
    <script src="compteurClics.js" async></script>
  </head>
  <body>
    <button id="clic">Cliquez-moi !</button>
    <p>Vous avez cliqué <span id="compteurClics">0</span> fois</p>
    <button id="desactiver">Désactiver le comptage</button></br></br>
  </body>
</html>
```

Qu'obtenez-vous ?

Que fait la fonction clic ?

Rajouter sur le fichier html un bouton Réactiver le comptage en-dessous du bouton Désactiver le compteur. Ajouter à la fin du fichier javascript, l'évènement qui réactive le compteur lors du clic sur le bouton Réactiver le compteur.

Sauver puis tester.

i) Les canvas :

L'élément HTML canvas est un élément qui va servir de conteneur et au sein duquel on va pouvoir dessiner toutes sortes de graphiques en utilisant le JavaScript. Il va servir de conteneur de dessins et figures.

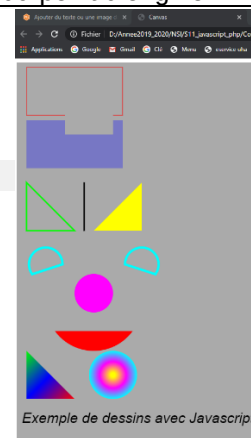
Pour dessiner dans un élément canvas, il faut suivre les étapes suivantes :

1. Accéder à l'élément canvas avec `document.getElementById()`
2. Accéder au contexte de rendu du canevas avec `getContext()`
3. Utiliser les propriétés et méthodes adaptées pour dessiner :

Méthode	Description
<code>strokeRect(x,y,l,h)</code>	Dessine dans le canvas un rectangle vide à la position x,y de longueur l et hauteur h
<code>strokeStyle</code>	Permet de définir la couleur du contour en hexadécimal ('#AABBCC')
<code>fillRect(x,y,l,h)</code>	Dessine dans le canvas un rectangle plein à la position x,y de longueur l et hauteur h
<code>fillStyle</code>	Permet de définir la couleur de remplissage en précisant soit les valeur rgb ou rgba si on veut de la transparence (de 0à1) Ex : <code>zone.fillStyle='rgba(255,0,0,0.5)'</code> Ou <code>zone.fillStyle='rgb(255,0,0)'</code>
<code>clearRect(x,y,l,h)</code>	Permet d'effacer une zone rectangulaire à la position x,y de longueur l et hauteur h
<code>beginPath()</code>	Permet de débiter un tracé sous forme de ligne
<code>moveTo(x,y)</code>	Permet de donner le point de départ du tracé à la position x,y
<code>lineTo(x,y)</code>	Permet de tracer une ligne de la position initiale à la position x,y
<code>lineWidth</code>	Permet de spécifier l'épaisseur de la ligne
<code>stroke()</code>	Permet d'appliquer le style <code>strokeStyle</code> (pour que la ligne soit visible)
<code>fill()</code>	Permet d'appliquer le remplissage <code>fillStyle</code>

closePath()	Permet de terminer le tracé.
arc(x,y,r,d1,f1,bool)	Permet de tracer un arc donc x correspond au décalage du point central de l'arc de cercle par rapport au bord gauche du canvas, y correspond au décalage du point central de l'arc de cercle par rapport au bord supérieur du canvas, r correspond au rayon, d1 correspond à l'angle de départ (en radians), f1 correspond à l'angle de fin (en radians) et bool indique si l'arc de cercle doit être dessiné dans le sens des aiguilles d'une montre (false, valeur par défaut) ou dans le sens inverse (true).
createLinearGradient(x1,y1,x2,y2)	Permet de créer un dégradé linéaire pour le remplissage où x1 est l'écart entre le point de départ du dégradé et le bord gauche du canevas, y1 est l'écart entre le point de départ du dégradé et le bord supérieur du canevas, x2 est l'écart entre le point de fin du dégradé et le bord gauche du canevas et y2 est l'écart entre le point de fin du dégradé et le bord supérieur du canevas .
addColorStop(d,'#couleur')	Permet d'ajouter des « couleurs stop », c'est-à-dire des points d'arrêt ou encore des transitions de couleurs dans notre dégradé. Ou d représente les points d'arrêt (entre 0 et 1) et #couleur la couleur en ce point en hexadécimal.
createRadialGradient(x1,y1,r1,x2,y2,r2)	Permet de créer un dégradé circulaire où x1 est l'écart entre le point de départ du dégradé et le bord gauche du canevas, y1 est l'écart entre le point (cercle) de départ du dégradé et le bord supérieur du canevas, r1 est le rayon du point (cercle) de départ du dégradé, x2 est l'écart entre le cercle de fin du dégradé et le bord gauche du canevas, y2 est l'écart entre le cercle de fin du dégradé et le bord supérieur du canevas et r2 est le rayon du cercle de fin du dégradé.
createPattern(img, motif)	Permet de mettre une image avant il faut charger l'image  ;  et motif : repeat : image répétée horizontalement et verticalement ; repeat-x : image répétée horizontalement uniquement ; repeat-y : image répétée verticalement uniquement ; no-repeat : l'image n'est pas répétée.
shadowOffsetX	Impose un décalage horizontal de l'ombre
shadowOffsetY	Impose un décalage vertical de l'ombre
shadowBlur	Permet de définir le flou Gaussien (plus la valeur sera grande, plus l'ombre sera étendue autour de la forme)
shadowColor	Permet d'indiquer le couleur de l'ombre en rgb ou rgba
strokeText('texte',x,y)	Affiche un texte creux à la position x,y
fillText('texte',x,y)	Affiche un texte plein à la position x,y
font('epaisseur,taille,police')	Permet de définir la police utilisée.
textAlign	Alignement qui peut être égal à start, end, left, right ou center par rapport au point de départ.
rotate(angle)	Permet de faire une rotation par rapport au point en haut à gauche. L'angle est donné en radians Math.PI/3 par exemple
translate(x,y)	x indique le déplacement horizontal du point d'origine tandis que le y indique le déplacement vertical du point d'origine

XXXIII)Créer un fichier dessin.js et un fichier JavaQ11.html et compléter grâce aux commentaires.
Saver puis tester. Vous devez obtenir l'image ci-contre.



```

let canvas=document.getElementById('c1');//récupère le canvas créer dans le
fichier html
let ctx=canvas.getContext('2d');//permet d'accéder à l'objet canvas.
ctx.strokeStyle='red';//Contour rouge
ctx.strokeRect(20,10,200,100);//Dessine un rectangle à 20,10,de longueur 200 et
hauteur 100
ctx.fillStyle='rgba(0,0,255,0.3)';//Remplissage bleu avec une transparence de 0.3
plus c'est bas plus c'est transparent
ctx.fillRect(20,120,200,100);//Dessine un rectangle à 20,120 de longueur 200 et hauteur
100
ctx.clearRect(100,50,100,100);//Efface une zone de 100x100 au point 100,50.

//commence un tracé
ctx.beginPath();
ctx.moveTo(20,250);//on se place au point 20,250
ctx.lineTo(20,350);//on trace une ligne jusqu'au point 20,350
... (20,350);//on trace une ligne jusqu'au point 120,350
... (120,350);//on trace une ligne jusqu'au point 20,250
ctx.strokeStyle='green';//Couleur verte
ctx.lineWidth = 3; //Epaisseur 3
ctx.closePath();//termine le tracé
ctx.stroke();//affiche le tracé

//commence un tracé
ctx.beginPath();
... (20,250);//on se place au point 140,250
... (140,250);//on trace une ligne jusqu'au point 140,350
... = 'black';//Couleur noire
... = 3; //Epaisseur 3
...;//termine le tracé
...;//affiche le tracé

//commence un tracé
ctx.beginPath();
... (160,350);//on se place au point 160,350
... (160,350);//on trace une ligne jusqu'au point 260,350
... (260,350);//on trace une ligne jusqu'au point 260,250
... (260,250);//on trace une ligne jusqu'au point 160,250
ctx.fillStyle='yellow';//Couleur jaune
...;//termine le tracé
ctx.fill();//affiche le tracé

//commence un tracé
ctx.beginPath();
ctx.strokeStyle='cyan';//Couleur cyan
ctx.lineWidth = 5; //Epaisseur 5
ctx.arc(60,420,35,0.8*Math.PI,2*Math.PI);//on trace un arc dont le centre est
à 60,420 de rayon 35 avec un angle de début 0.8PI et angle de fin 2 pi sens
horaire
...;//termine le tracé
...;//affiche le tracé

//commence un tracé
ctx.beginPath();
ctx.strokeStyle='cyan';//Couleur cyan
ctx.lineWidth = 5; //Epaisseur 5
ctx.arc(260,420,35,0.2*Math.PI,Math.PI,true);//on trace un arc dont le centre est à 260,420 de rayon
35 avec un angle de début 0.2PI et angle de fin pi sens trigo
...;//termine le tracé
...;//affiche le tracé

//commence un tracé
ctx.beginPath();
ctx.fillStyle='magenta';//Couleur magenta

```

```

ctx.arc(...,...,...); //on trace un arc dont le centre est à 160,480 de rayon 40
avec un angle de début 0 et angle de fin 2pi sens horaire
...; //termine le tracé
...; //affiche le tracé

//commence un tracé
ctx.beginPath();
ctx.fillStyle='...'; //Couleur rouge
ctx.lineWidth = ...; //Epaisseur 5
ctx.arc(...,...,...); //on trace un arc dont le centre est à 160,500 de rayon 100
avec un angle de début 0.2PI et angle de fin 0.8pi sens horaire
...; //termine le tracé
...; //affiche le tracé

//dégradé lineaire
let lineaire=ctx.createLinearGradient(...,...,...); //dégradé qui partira du point
20,600 et qui ira au point 120,700
lineaire.addColorStop(0,'...'); //dégradé qui commence au vert
lineaire.addColorStop(0.5,'...'); //dégradé qui aura du bleu au milieu
lineaire.addColorStop(1,'...'); //dégradé qui termine avec du rouge

//commence un tracé
ctx.beginPath();
... (...,...); //on se place au point 20,600
... (...,...); //on trace une ligne jusqu'au point 20,700
... (...,...); //on trace une ligne jusqu'au point 120,700
... (...,...); //on trace une ligne jusqu'au point 20,600
ctx.fillStyle=lineaire; //Couleur lineaire (gradient créer au-dessus)
...; //termine le tracé
...; //affiche le tracé

// dégradé radial
radial=ctx.createRadialGradient(200,650,5,200,650,50); //dégradé radial de
centre 200,650 et de premier rayon 5 et dernier rayon 50
radial.addColorStop(0,'...'); //dégradé qui commence au jaune
radial.addColorStop(0.5,'...'); //dégradé qui aura du magenta au milieu
radial.addColorStop(1,'...'); //dégradé qui termine avec du cyan
//commence un tracé
ctx.beginPath();
ctx.fillStyle=radial; //Couleur radial (gradient créer au-dessus)
ctx.arc(...,...,...); //on trace un arc dont le centre est à 200,650 de rayon 50
avec un angle de début 0 et angle de fin 2pi sens horaire
...; //termine le tracé
...; //affiche le tracé

//Ajout de texte
ctx.font='Italic 30px Arial'; //Texte Arial 30 pixels en italic
ctx.fillStyle='...'; //couleur noire
ctx.fillText(...,...,...); //affichage du texte Exemple de dessins en Javascript au
point 10,750.

```

Java11.html

```

<!doctype html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Canvas</title>
    <script src="dessin.js" async></script>
  </head>
  <body>
    <canvas id='c1' style='background-color:#AAAAAA' width="500"
height='800';></canvas>
  </body>
</html>

```

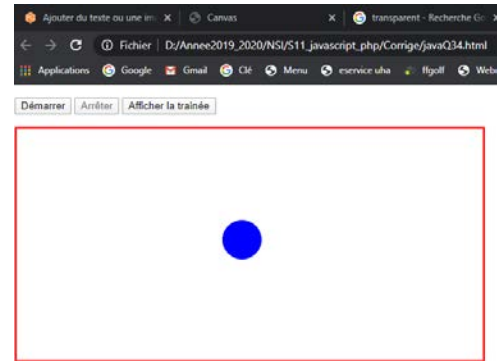

j) Animation :

Commençons par créer une page html qui comprend trois boutons :

- demarrer
- arrêter
- trainee

ainsi qu'un canvas de largeur 600 * 300 pour cela compléter le code ci-dessous et sauvegarder sous java12.html.

La balle sera gérée par le fichier Javascript ballon.js



```
<!doctype html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Ballon rebondissant</title>
    <script src="ballon.js" async></script>
  </head>
  <body>
    <p>
      <button id=".....">Démarrer</button>
      <button id="....." disabled>Arrêter</button>
      <button id=".....">Afficher la trainée</button>
    </p>
    <canvas id="canvas" width="..." height="..."></canvas>
  </body>
</html>
```

Créer le fichier ballon.js à l'aide du script ci-dessous. On vous demande de remplacer les et de commenter chaque ligne qui ne l'est pas.

Sauvegarder puis tester.

```
let canvas = document.getElementById('canvas');
let ctx = canvas.getContext('2d');
let raf;
let trainee=false;

let ball = { //on crée l'objet ball
  x: 100,
  y: 100,
  vx: 5,
  vy: 2,
  radius: 25,
  color: 'blue',
  draw: function() {
    ctx.beginPath();
    ctx.arc(this.x, this.y, this.radius, 0, Math.PI*2, true);
    ctx.closePath();
    ctx.fillStyle = this.color;
    ctx.fill();
  }
};

function clear(){ //création de la fonction clear
  if (trainee){
    ctx.fillStyle = 'rgba(255,255,255,0.2)';
  }
  else
  {
    ctx.fillStyle = 'rgb(255,255,255)';
  }
  ctx.fillRect(0,0,canvas.width,canvas.height);
  ctx.strokeStyle='rgb(255,0,0)';
  ctx.lineWidth = 5;
  ctx.strokeRect(0,0,canvas.width,canvas.height);
}

function draw() { //création de la fonction draw
```

```

clear();
ball.draw();
ball.x += ball.vx;
ball.y += ball.vy;
raf = window.requestAnimationFrame(draw)
    if (ball.y + ball.vy+ball.radius > canvas.height || ball.y + ball.vy-
ball.radius < 0) {
        ball.vy = -ball.vy;
    }
    if (ball.x + ball.vx+ball.radius > canvas.width || ball.x + ball.vx-
ball.radius < 0) {
        ball.vx = -ball.vx;
    }
}

let demarrerBtn = document.getElementById("...");//Récupération du bouton
demarrer
let arreterBtn = document.getElementById("..."); //Récupération du bouton arreter
let traineeBtn = document.getElementById("..."); //Récupération du bouton trainee

demarrerBtn.addEventListener("click", function () {
    // Change l'état des boutons
    demarrerBtn.disabled = true;
    arreterBtn.disabled = false;
    // Démarre l'animation
    raf = window.requestAnimationFrame(draw);
});

arreterBtn.addEventListener("click", function () {
    // Change l'état des boutons
    demarrerBtn.disabled = false;
    arreterBtn.disabled = true;
    // Arrête l'animation
    window.cancelAnimationFrame(raf);
});

traineeBtn.addEventListener("click", function () {
    if (trainee== true)
    {
        trainee=false;
        document.getElementById("trainee").innerHTML = "Afficher la trainée";
    }
    else
    {
        trainee=true;
        document.getElementById("trainee").innerHTML = "Désactiver la
trainée";
    }
});

ball.draw();

```